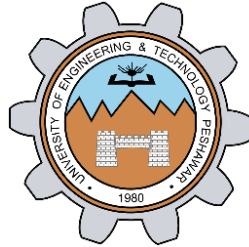


Interfacing Seven Segment Display to 8051 Development kit

LAB # 05



Spring 2024

CSE-307L Microprocessor Based System Design Lab

Submitted by: **Ali Asghar**

Registration No.: **21PWCSE2059**

Class Section: **C**

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Submitted to:

Dr. Asif Ali Khan

Date:

29th March 2024

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Task 1:

Run all the programs given in the lecture

Code:

```
task1.c STARTUP.A51
1  #include<reg51.h>
2  void delay();
3
4  void main(void){
5
6      while(1){
7          P0 = 0x00;
8
9          for(;;){
10             P0 = 0xF9;
11             delay();
12         }
13     }
14 }
15 void delay(){
16     unsigned int y;
17     for(y = 0; y<30000; y++){
18     }
19 }
20
```

Output:

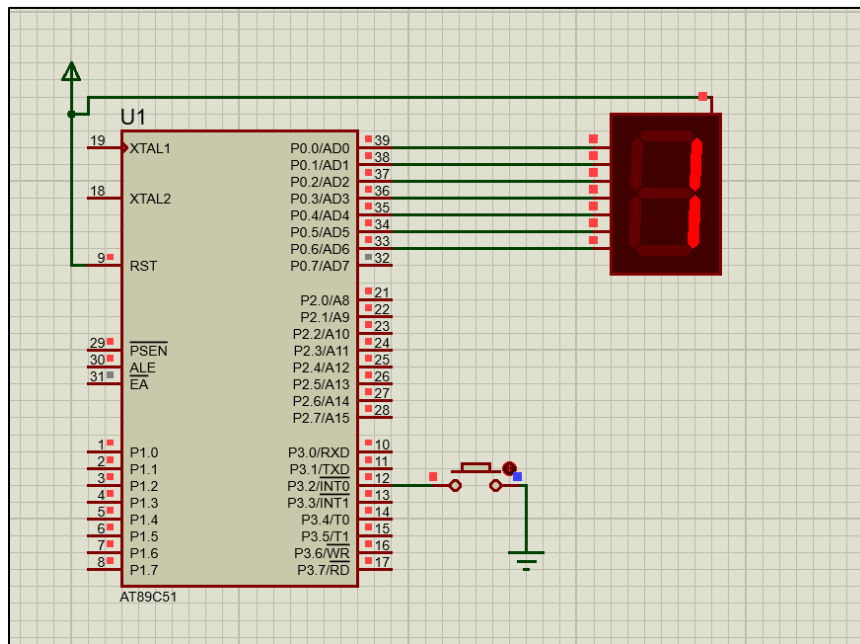
The screenshot displays the Keil IDE interface. On the left, the 'Registers' window shows the status of various registers. The 'R7' register is highlighted with a value of 0x0d. The 'Disassembly' window on the right shows the assembly code for the program, with the 'while' loop and 'delay' function being the current focus. The assembly code is as follows:

```
18:      }
C:0x0803 0F INC R7
C:0x0804 BF0001 CJNE R7,#0x00,C:0808
C:0x0807 0F TNC R6
```

The 'Registers' window shows the following values:

Register	Value
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x0d
sys	
a	0x00
b	0x00
sp	0x09
sp_max	0x09
dptr	0x000
PC	0x00
states	463
sec	0.000
psw	0x80

Proteus Simulation:



Task 2:

Display different digits on the Seven Segment Display

Code:

```
1  #include<reg51.h>
2  void delay();
3
4  void main(void) {
5
6      while(1) {
7          P0 = 0x40; //0
8          delay();
9          P0 = 0x79; //1
10         delay();
11         P0 = 0x24; //2
12         delay();
13         P0 = 0x30; //3
14         delay();
15         P0 = 0x19; //4
16         delay();
17         P0 = 0x12; //5
18         delay();
```

```

16     delay();
17     P0 = 0x12; //5
18     delay();
19     P0 = 0x02; //6
20     delay();
21     P0 = 0x78; //7
22     delay();
23     P0 = 0x00; //8
24     delay();
25     P0 = 0x18; //9
26 }
27 }
28 void delay(){
29     unsigned int y, x;
30     for(x = 0; x<30000; x++);
31     for(y = 0; y<30000; y++);
32 }
33

```

Output:

The screenshot shows a debugger window with two panes. The top pane, titled 'Disassembly', shows the following instructions:

Address	Disassembly	Comment
9:		P0 = 0x79; //1
C:0x0806	758079 MOV	P0(0x80),#0x79
10:		delay();
C:0x0809	12083B LCALL	delay(C:083B)

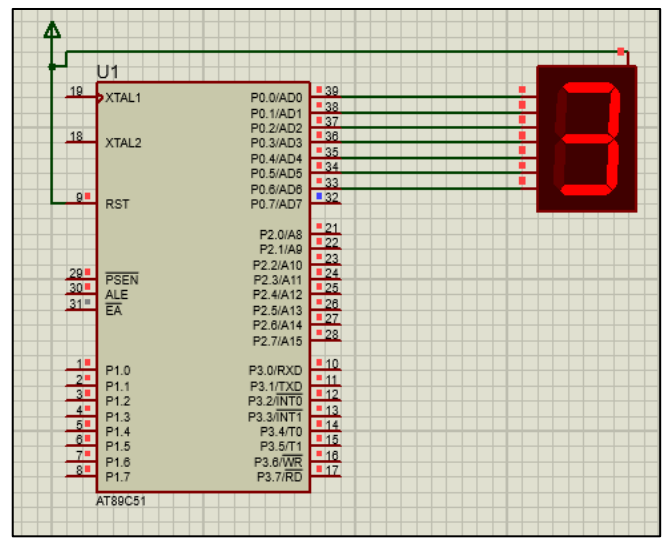
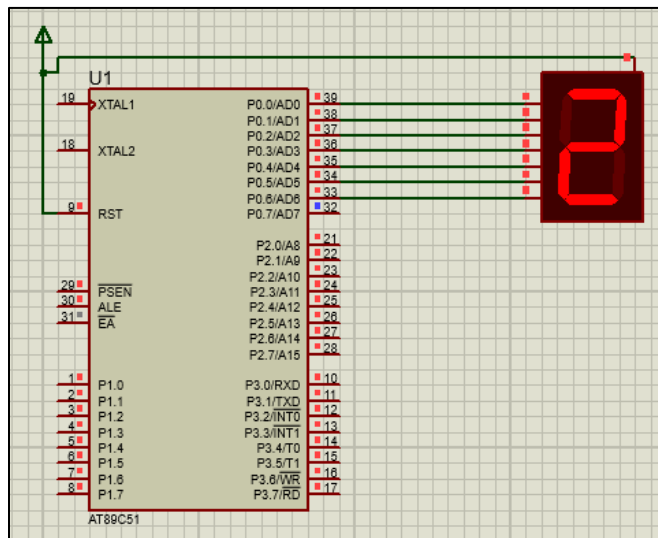
The bottom pane shows the source code for 'task3.c' with line numbers 4 through 19. The code is as follows:

```

4 void main(void){
5
6     while(1){
7         P0 = 0x40; //0
8         delay();
9         P0 = 0x79; //1
10        delay();
11        P0 = 0x24; //2
12        delay();
13        P0 = 0x30; //3
14        delay();
15        P0 = 0x19; //4
16        delay();
17        P0 = 0x12; //5
18        delay();
19        P0 = 0x02; //6

```

Proteus Simulation:



Task 3:

Display digits from 0 to F on Seven Segment Display

Code:

```

1  #include<reg51.h>
2  void delay();
3
4  void main(void) {
5
6      while(1) {
7          P0 = 0x40; //0
8          delay();
9          P0 = 0x79; //1
10         delay();
11         P0 = 0x24; //2
12         delay();
13         P0 = 0x30; //3
14         delay();
15         P0 = 0x19; //4
16         delay();
17         P0 = 0x12; //5
18         delay();
19         P0 = 0x02; //6
20         delay();
21         P0 = 0x78; //7
22         delay();
23         P0 = 0x00; //8
24         delay();

```

```

24     delay();
25     P0 = 0x18; //9
26     delay();
27     P0 = 0x08; //A
28     delay();
29     P0 = 0x00; //B
30     delay();
31     P0 = 0x46; //C
32     delay();
33     P0 = 0x40; //D
34     delay();
35     P0 = 0x06; //E
36     delay();
37     P0 = 0x0E; //F
38     delay();
39
40 }
41 }
42 void delay(){
43     unsigned int y, x;
44     for(x = 0; x<30000; x++);
45     for(y = 0; y<30000; y++);
46 }
47

```

Output:

The screenshot displays a disassembler window at the top and an IDE window below it. The disassembler shows assembly instructions for the `delay()` function, including `MOV P0(0x80), #0x24` and `LCALL delay(C:0862)`. The IDE window shows the C source code for `task2.c`, with lines 8 through 21 visible. A 'Parallel Port 0' window is open, showing the current port value as `0x79` and the pins as `0x40`.

Disassembly

```

12:      delay();
C:0x080C 758024 MOV     P0(0x80), #0x24
13:      P0 = 0x30; //3
C:0x080F 120862 LCALL  delay(C:0862)

```

task2.c

```

8      delay();
9      P0 = 0x79; //1
10     delay();
11     P0 = 0x24; //2
12     delay();
13     P0 = 0x30; //3
14     delay();
15     P0 = 0x19; //4
16     delay();
17     P0 = 0x12; //5
18     delay();
19     P0 = 0x02; //6
20     delay();
21     P0 = 0x78; //7

```

Parallel Port 0

Port 0: 0x79 7 Bits 0

Pins: 0x40

Proteus Simulation:

